

Contents

Logic Implemented This Engagement — Field Mapping & Formula Reference (TSRe → ILSTerms)	1
0.2 Company/Industry Loss Minimum Fallback	1
1. Franchise Deductible	2
2. Reverse Franchise Deductible	2
3. Inner Aggregate Deductible	3
4. Outer Aggregate Deductible	3
5. Reinstatement Premium Waterfall (CATXOL/RPP/CATBOND)	3
6. CATBOND Unlimited-Occurrence-Limit Cap	4
7a. RPP Swap Mechanism	4
7b. RPP Rate Lookup Fix	5
8. AGGTRIGGER (Annual ILW)	5
9. OCCTRIGGER (Per-Occurrence ILW, Nth Event)	5
10. PRORATA	6
11. OCCILW (Occurrence-Capped ILW with Proportional Scaling)	6
12. Quota Share Ceded % — BLOCKED	7
13. Coinsurance — BLOCKED	7
14. ELSE Catch-All / GU Layers	7
15. Drop-Down Cover	8
Structural Confirmation: Multiple Children at One Parent (Sourcing vs. Inuring)	8
Fields With No TSRe Source At All	9

Logic Implemented This Engagement — Field Mapping & Formula Reference (TSRe → ILSTerms)

Every fix/logic implemented in arapl_RunILSAnalysis_SR this engagement: the exact formula as deployed and tested, the ILSTerms field(s) it reads, and the corresponding TSRe source field. Field mappings sourced from earlier engagement work (TSRe_to_RiskAnalyzer_Field_Mapping_Comparison.md, Reinsurance_Types_Mapped_to_TSRe_Fields.md) — not reconstructed from memory. Formulas are the actual deployed logic, verified against real or synthetic test data this engagement (see the full test log for the specific numbers).

Notation: min/max used in the standard sense; Σ = sum across events in the stated scope (usually a year).

0.2 Company/Industry Loss Minimum Fallback

Formula (as implemented):

EffectiveLossMinimum = CompanyLossMinimum if CompanyLossMinimum > 0
= IndustryLossMinimum otherwise

EffectiveLossMaximum = CompanyLossMaximum if CompanyLossMaximum < DEFAULT_UNLIMITED
= IndustryLossMaximum otherwise

Every trigger check downstream (items 1-4, 8-11) uses EffectiveLossMinimum/Maximum in place of the Industry-only values used before this fix.

ILSTerms field	TSRe source	Wired end-to-end?
CompanyLossMinimum, CompanyLossMaximum	tRT.CompanyLossMinimum, tRT.CompanyLossMaximum	Yes

ILTerms field	TSRe source	Wired end-to-end?
IndustryLossMinimum, IndustryLossMaximum	tRT.IndustryLossMinimum, tRT.IndustryLossMaximum	Yes

Status: confirmed working — 137→133 real events, exactly matching the 4 events below the \$75M threshold.

1. Franchise Deductible

Formula (as implemented, matches the documented TSRe-side mechanic):

Contract Loss = 0 if Loss < EffectiveLossMinimum
Contract Loss = Loss if Loss ≥ EffectiveLossMinimum

(Documented form: Contract Loss = 0 if Loss < tRT.CompanyLossMinimum; else min(Loss, tRT.OccurrenceLimit) — the occurrence limit is applied in a later, separate step in this engine, not combined into this filter.)

ILTerms field	TSRe source	Wired end-to-end?
DeductibleType (bit 1)	None — RiskAnalyzer-only concept	No — never populated by ARAPL_populate_ILTerms
CompanyLossMinimum/IndustryLossMinimum (via EffectiveLossMinimum)	tRT.CompanyLossMinimum, tRT.IndustryLossMinimum	Yes

Status: confirmed working on real data (manually-set DeductibleType, since it's never populated from TSRe).

2. Reverse Franchise Deductible

Formula (as implemented, matches the documented TSRe-side mechanic):

Contract Loss = Loss if Loss ≤ EffectiveLossMaximum
Contract Loss = 0 if Loss > EffectiveLossMaximum

Combined with Franchise (both bits set):

Include event only if: (Franchise OFF OR Loss ≥ EffectiveLossMinimum)
AND (Reverse OFF OR Loss ≤ EffectiveLossMaximum)

ILTerms field	TSRe source	Wired end-to-end?
DeductibleType (bit 2)	None	No
CompanyLossMaximum/IndustryLossMaximum	tRT.CompanyLossMaximum, tRT.IndustryLossMaximum	Yes

Status: confirmed working — verified surviving losses correctly bounded on both ends (\$75M-\$900M in testing).

3. Inner Aggregate Deductible

Formula (as implemented, matches the documented “Otherwise Recoverable” mechanic):

AggregateRetention = AggregateDeductible (when this bit is set)
 NetNetLoss = min(AggregateLimit, max(0, tmpNetLoss - AggregateRetention))

ILSTerms field	TSRe source	Wired end-to-end?
DeductibleType (bit 4)	None	No
AggregateDeductible	None — RiskAnalyzer-only concept per field-mapping work	No

Status: confirmed working — exact match on all 40 non-zero real test years.

4. Outer Aggregate Deductible

Formula (as implemented, matches the documented “two linked programs” mechanic, collapsed into one field here):

NetLoss(year) = max(0, $\sum$ Loss(year) - AggregateDeductible)
 Each event's Contract Loss = Event Loss $\times$ NetLoss(year) / $\sum$ Loss(year)
 [proportional scaling]

ILSTerms field	TSRe source	Wired end-to-end?
DeductibleType (bit 8)	None	No
AggregateDeductible	None	No

Status: confirmed working. **Finding:** this bit is mutually exclusive with bits 1/2 in the current code — if set, Franchise/Reverse Franchise are silently skipped regardless of their own bits.

5. Reinstatement Premium Waterfall (CATXOL/RPP/CATBOND)

Formula (as implemented — a banded/tiered structure, consistent with the documented Reinstatement Premium = Premium $\times$ (Limit Reinstated / OccurrenceLimit) mechanic, but expressed per-band):

For reinstatement band i (1-indexed), with rate R_i and cumulative prior charge P_{i-1} ($P_0 = 0$):
 Band boundary(i) = $i \times \text{OccurrenceLimit}$

If ($\sum \text{Loss} - \text{AggregateRetention}$) $\leq$ Band boundary(i):
 ReinstatementPremium = $P_{i-1} + (R_i \times \text{ROL})$
 $\times [(\sum \text{Loss} - \text{AggregateRetention}) - \text{Band boundary}(i-1)]$

Else (all bands exhausted):
 ReinstatementPremium = $\sum_i (R_i \times \text{ROL} \times \text{OccurrenceLimit})$ [flat ceiling]

$P_i = P_{i-1} + (R_i \times \text{ROL} \times \text{OccurrenceLimit})$

Where ROL = PercentAnnualSpread (or Premium / PrincipalAmount if blank).

ILSTerms field	TSRe source	Wired end-to-end?
ReinstatementPremium (the "N@Rate,N@Rate" string, parsed into bands/rates)	tRT.ReinstatementPremium (also tReinsuranceTreatyReinstatement.ReinstatementPremium)	Yes
PercentAnnualSpread, Premium, PrincipalAmount	tRT (identical names)	Yes
OccurrenceLimit	tRT.OccurrenceLimit	Yes

Status: confirmed working — all bands and the flat ceiling verified exact on real + synthetic data.

6. CATBOND Unlimited-Occurrence-Limit Cap

Formula (as implemented — proposed fix):

$$\text{OccurrenceLimit_effective} = \text{PrincipalAmount}$$
 (only when OccurrenceLimit is blank/unlimited)

$$\text{Contract Loss} = \min(\max(0, \text{Loss} - \text{OccurrenceRetention}), \text{OccurrenceLimit_effective})$$
 (Matches the documented CAT bond mechanic — Principal Reduction = min(Loss, PrincipalAmount) — which this fix now correctly enforces; previously, OccurrenceLimit stayed at the unlimited placeholder, so this cap never applied.)

ILSTerms field	TSRe source	Wired end-to-end?
OccurrenceLimit,	tRT.OccurrenceLimit,	Yes
OccurrenceRetention	tRT.OccurrenceRetention	
PrincipalAmount	tRT.PrincipalAmount	Yes

Status: bug confirmed (\$625M modeled vs \$250M real principal); fix mechanically confirmed.
Pending business sign-off — no CAT bond chapter in the reference documentation to validate this specific rule against.

7a. RPP Swap Mechanism

Formula (as implemented):

For the container layer (the second-to-last-processed layer in an RPP structure):

$$\text{Contract Loss} = \text{Event Loss} / \Sigma \text{Loss}(\text{year}) \times \text{ReinstatementPremium}(\text{year})$$
 (Replaces the layer's normal net-loss output with a share of the *protected layer's* reinstatement premium — this is the actual "protection" payout.)

ILSTerms field	TSRe source	Wired end-to-end?
InuringSequenceNumber, ParentRelationType (= 4, marks the container)	InuringSequenceNumber from tRT; ParentRelationType has no TSRe source	Partial — sequence number wired, relationship marker is not

Status: confirmed working, including correct proportional back-allocation across multi-event years.

7b. RPP Rate Lookup Fix

Formula (as implemented — fix):

RPPROL = LayerPremium(container layer's row) / OccurrenceLimit(protection layer)
ROL (used in item 5's formula above) = RPPROL, for ReinsuranceTypeCode = 'RPP'

(Previously, RPPROL was read from the *protection layer's own row*, which structurally can never satisfy the lookup condition — always resolving to 0, silently zeroing every reinstatement premium calculation for RPP layers.)

ILSTerms field	TSRe source	Wired end-to-end?
LayerPremium	None — RiskAnalyzer-only concept	No — confirmed NULL on every real layer in the database

Status: fix mechanically confirmed (test rate \$2M → 4% ROL, all banded values scaled correctly).
No real RPP structure exists to validate against.

8. AGGTRIGGER (Annual ILW)

Formula (as implemented, matches the documented ILW mechanic with a binary payout):

Contract Loss(year) = PrincipalAmount if $\Sigma \text{Loss}(\text{year}) > \text{AggregateRetention}$
Contract Loss(year) = 0 otherwise

ILSTerms field	TSRe source	Wired end-to-end?
AggregateRetention	tRT.AggregateRetention	Yes
PrincipalAmount	tRT.PrincipalAmount	Yes
CompanyLossMinimum/IndustryLossMinimum, (event filter, via EffectiveLossMinimum)	tRT.CompanyLossMinimum, tRT.IndustryLossMinimum	Yes

Status: confirmed working — clean binary split across all 140 test years, no partial values.

9. OCCTRIGGER (Per-Occurrence ILW, Nth Event)

Formula (as implemented — Method B from the documented Nth-Event mechanic):

Event Sequence = ROW_NUMBER() of events in the year, ordered by Day,
restricted to events where Loss > EffectiveLossMinimum

Contract Loss = PrincipalAmount for the event where Event Sequence = StartingEventNumber
Contract Loss = 0 for every other event

ILTerms field	TSRe source	Wired end-to-end?
StartingEventNumber	tRT.StartingEventNumber	Yes — was already wired before this engagement, but flagged as unused by the calculation; this engagement's testing confirms it is now correctly read in this branch
PrincipalAmount, CompanyLossMinimum	tRT.PrincipalAmount, tRT.CompanyLossMinimum	Yes

Status: confirmed working — on the one real year with two qualifying events, correctly skipped the 1st and paid only the 2nd.

10. PRORATA

Formula (as implemented — fix corrects the retention-offset term):

AggregateRetention_effective = AggregateRetention + (EventNumber - 1) × AggregateLimit
 [FIX: was × OccurrenceLimit, which this branch hardcodes to "unlimited," making the offset unreachable whenever StartingEventNumber ≥ 2]

NetLoss(year) = min(AggregateLimit, max(0, ΣLoss(year) - AggregateRetention_effective))

Contract Loss(event) = Event Loss × NetLoss(year)/ΣLoss(year)
 × (PrincipalAmount / AggregateLimit)

(Matches the documented Quota-Share-style proportional mechanic, Contract Loss = min(Participation × Loss, Limit), generalized here as a ratio-based tranche share.)

ILTerms field	TSRe source	Wired end-to-end?
AggregateLimit, AggregateRetention	tRT.AggregateLimit, tRT.AggregateRetention	Yes
PrincipalAmount	tRT.PrincipalAmount	Yes
StartingEventNumber	tRT.StartingEventNumber	Yes (wired, but previously non-functional in this branch due to the bug above)

Status: bug confirmed (paid \$0 regardless of loss size, with StartingEventNumber ≥ 2); fix confirmed exact across all 140 real test years. **Pending business sign-off** — no real PRORATA layer or documentation exists to validate against.

11. OCCILW (Occurrence-Capped ILW with Proportional Scaling)

Formula (as implemented — combines Per-Occurrence XOL with proportional scaling):

OccLoss(event) = min(max(0, Event Loss - OccurrenceRetention), OccurrenceLimit)
 NetLoss(year) = min(AggregateLimit, max(0, ΣOccLoss(year) - AggregateRetention))
 × (PrincipalAmount/AggregateLimit)

Contract Loss(event) = OccLoss(event) × NetLoss(year)/ΣOccLoss(year)

ILSTerms field	TSRe source	Wired end-to-end?
OccurrenceLimit, OccurrenceRetention	tRT.OccurrenceLimit, tRT.OccurrenceRetention	Yes
AggregateLimit, PrincipalAmount	tRT.AggregateLimit, tRT.PrincipalAmount	Yes

Status: confirmed working — verified the aggregate-cap-then-scale order is applied correctly on a multi-event test year.

12. Quota Share Ceded % — BLOCKED

Documented formula (TSRe side, not currently implemented in this engine):

Contract Loss(event) = min(ParticipationGross × Company Loss, OccurrenceLimit)
Contract Loss(year) = min(Σ above, AggregateLimit)

ILSTerms field	TSRe source	Wired end-to-end?
QS_Cession <i>(possible unconfirmed source)</i>	None found — flagged as RiskAnalyzer-only tRT.CededAmount, or tAp- pliesToAreaGeoXRef.CededPerce nt if geography-specific	No — this is the entire blocker Unconfirmed — needs direct confirmation before treating as the fix

Status: blocked — no cession % is applied anywhere currently; needs a confirmed source field before any formula can be implemented.

13. Coinsurance — BLOCKED

Documented formula (TSRe side, not currently implemented in this engine):

Reinsurer's Share = Layer Loss × (1 – CoinsuranceAmount)

ILSTerms field	TSRe source	Wired end-to-end?
CoinsuranceAmount	tRT.CoinsuranceAmount	Yes — the field itself is wired

Status: blocked — not on data, but on interpretation. The documented formula implies CoinsuranceAmount = the **cedant's retained share** (since it's subtracted from 1), but this hasn't been explicitly confirmed for this specific implementation — applying it backwards would invert every affected result.

14. ELSE Catch-All / GU Layers

Formula (as implemented — plain pass-through, no terms applied):

Contract Loss = Gross Loss (unmodified)

ILSTerms field	TSRe source	Wired end-to-end?
ReinsuranceTypeCode (= 'GU')	Not checked against TSRe's own type-coding this engagement	Unknown
ParentILSTermsID, ParentRelationType (= 3, GU-to-CATBOND link)	None — RiskAnalyzer-only	No

Status: confirmed correct as a pass-through mechanism. Real GU layers found with no loss data connected — reflects this working database's limited data, not confirmed as a production gap.

15. Drop-Down Cover

Formula (as implemented — newly built this engagement):

TriggersPrimary(event) = 1 if Event Loss > PrimaryOccRetention, else 0
PriorUses(event) = $\sum$ TriggersPrimary for all earlier events in the same year (by Day)

If PriorUses $\leq$ PrimaryReinstatement:

Contract Loss = min(max(0, Loss - PrimaryOccRetention), PrimaryOccLimit)

Else (primary capacity used up):

Contract Loss = min(max(0, Loss - StepOccRetention), StepOccLimit)

(Matches the documented mechanic exactly: Layer 1: Recovery = min(max(0, Loss-Retention₁), Limit₁); Step-Layer: Recovery = min(max(0, Loss-Retention₂), Limit₂), activates once Layer 1 is exhausted — verified against AIR's own 4-event worked example.)

ILSTerms field	TSRe source	Wired end-to-end?
StepLayerSequenceNumber	tRT.StepLayerSequenceNumber — confirmed to exist on TSRe	No — ARAPL_populate_ILSTerms never carries it across; no column exists in production ILSTerms at all (test-only column added this engagement)
OccurrenceRetention, OccurrenceLimit (both layers)	tRT.OccurrenceRetention, tRT.OccurrenceLimit	Yes
ParentRelationType (= 5, new value introduced this engagement)	None — newly invented, RiskAnalyzer-side only	No, by design

Status: confirmed working, exact match to AIR's documented example. **Three separate pieces of upstream work still needed** before any real deal reaches this logic: (1) add the column to production ILSTerms, (2) update ARAPL_populate_ILSTerms to carry tRT.StepLayerSequenceNumber, (3) this engagement's engine logic (already built and validated).

Structural Confirmation: Multiple Children at One Parent (Sourcing vs. In-uring)

Formula (as implemented, applies to any parent-child relationship — RPP, GU, Drop-Down, or plain cascading):

For each child with ParentRelationType = 1 (sourced):
 Parent receives = Child's own Contract Loss, unmodified

For each child with ParentRelationType = 2 (inuring):
 Parent receives = Child's Gross Loss – Child's own Contract Loss [the residual]

Parent's total intake = Σ (all children's contributions,
 per the rule above, each independently)

ILSTerms field	TSRe source	Wired end-to-end?
ParentILSTermsID	Represented in TSRe via tReinsuranceProgramSourceInure (a relationship table, not a single field)	Structurally different — TSRe expresses this as a program relationship, not a direct field-to-field mapping
ParentRelationType	None — RiskAnalyzer-only	No

Status: confirmed working — built and tested a parent with 3 children (2 sourced, 1 inuring); each contributed correctly and independently.

Fields With No TSRe Source At All

Field	Used by	Implication
DeductibleType	Items 1–4	Root cause of Step 0.1 — blocks all deductible logic on real converted data
AggregateDeductibleLayerPremium	Items 3–4 Item 7b	Same as above No real RPP structure has ever used this field
ParentRelationType	Items 7, 14, 15, and the multi-child structural test	Every multi-layer linkage tested this engagement depends on a field with no TSRe source
QS_Cession	Item 12	Entire blocker for this item

Pattern worth flagging on its own: every “blocked” or “pending sign-off” item traces back to a field in this list. The calculation logic itself is, at this point, largely confirmed correct across every tested branch — **the remaining risk is almost entirely in data population, not calculation logic.**